PRACTICAL NO.4(E)

Aim:- Display a list of items sorted in the state using map().

→ In React, map() is a standard JavaScript array method used to loop over an array of data and convert it into an array of HTML elements or components.

Code:-

App.jsx:-

```
import React, { useState } from "react";

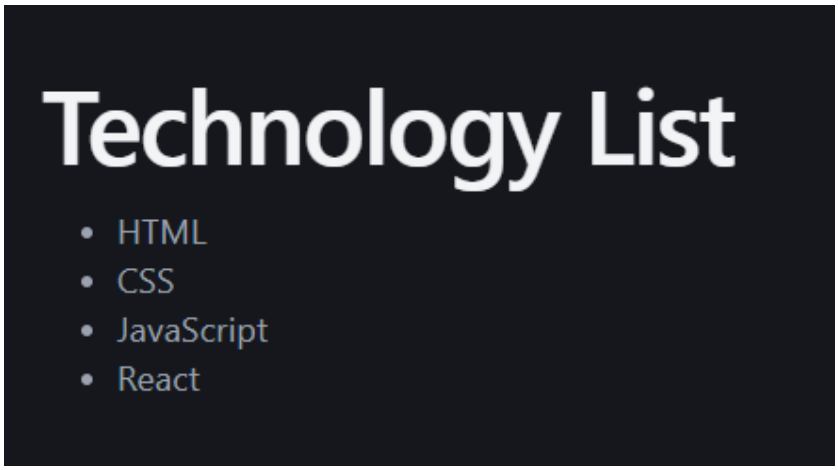
function App() {
  const [items, setItems] = useState([
    "HTML",
    "CSS",
    "JavaScript",
    "React"
  ]);

  return (
    <div style={{ textAlign: "left", padding:"20px" }}>
      <h1>Technology List</h1>

      <ul>
        {items.map((item, index) => (
          <li key={index}>{item}</li>
        ))}
      </ul>
    </div>
  );
}

export default App;
```

Output:-



PRACTICAL NO.4(F)

Aim:- Build a samll interactive component.(Eg:- Counter or to-do list)

Code:-

App.jsx:-

```
import React, { useState } from "react";

function Counter(){
  const [count,setCount]=useState(0);

  return(
    <div>
      <h1>Counter Application</h1>

      <h2>Count:{count}</h2>

      <button onClick={()=>setCount(count+1)}>
        Increase
      </button>

       <button onClick={()=>setCount(count-1)}>
```

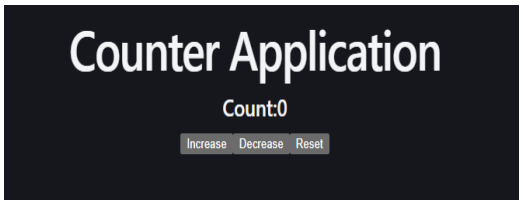
```
        Decrease
      </button>

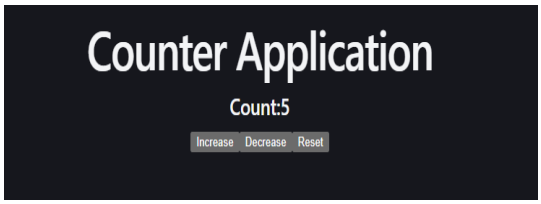
      <button onClick={()=>setCount(0)}>
        Reset
      </button>
    </div>
  );
}

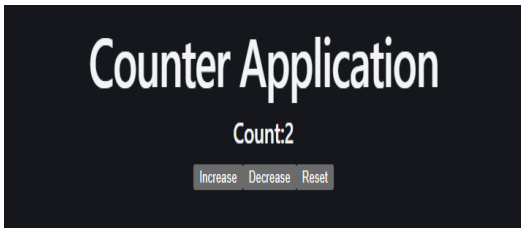
function App(){
  return(
    <Counter/>
  );
}

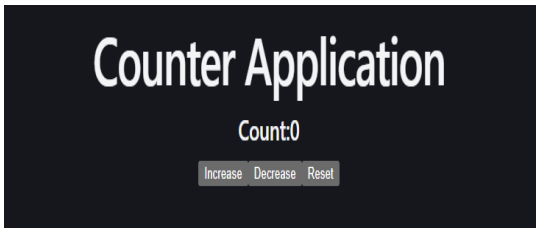
export default App;
```

Output:-









PRACTICAL 2. Core JavaScript Concept for MERN

PRACTICAL NO.2(A)

Aim:- Create a function and call them with parameter.

Code:-

```
//Function with Parameters
function greet(name){
   console.log("Hello,"+name);
}

//Function Call
greet("John");
greet("Alice")
```

Output:-

```
Hello,John
Hello,Alice
```

PRACTICAL NO.2(B)

Aim:- Work with arrays and object in JavaScript.

Code:-

```
//Array
let fruits=["Apple","Banana","Mango"];

//Object
let students={
   name:"John",
   age:20,
   course:"TYBSc IT"
};
```

```
console.log("Fruits:",fruits);
console.log("Student Name:",students.name);
console.log("Age:",students.age);
console.log("Course:",students.course);
```

Output:-

Fruits: ['Apple', 'Banana', 'Mango']
Student Name: John
Age: 20
Course: TYBSc IT

Push & Pop Array Method:-

Code:-

```
let students=["Amit","Sneha","Rohan"];

//Array Methods
students.push("Priya");
students.pop();

console.log("Students List:");
console.log(students);

let Student={
   rollNo: 101,
   name: "Amit",
   course: "TYIT"
};

console.log("\nStudents Details");
console.log("Student RollNo:",Student.rollNo);
console.log("Student Name:",Student.name);
console.log("Student Course:",Student.course);
```

Output:-

Students List:
['Amit', 'Sneha', 'Rohan']

Students Details:-
Student RollNo: 101
Student Name: Amit
Student Course: TYIT

PRACTICAL NO.2(C)

Aim:- Demonstrate use of arrow function.

Code:-

```
//Arrow Function
const add=(a,b)=>a+b;

console.log("Sum=",add(10,20));
```

Output:-

```
Sum= 30
```

PRACTICAL NO.2(D)

Aim:- Write a program using array methods (map, filter, reduce).

Code:-

```
let numbers=[10,20,30,40,50];

let doubled=numbers.map(num=>num*2)
let greaterThan25=numbers.filter(num=>num>25);
let total=numbers.reduce((sum,num)=>sum+num,0);

console.log("Original:",numbers);
console.log("Map:",doubled);
console.log("Filter:",greaterThan25);
```

```
console.log("Reduce:",total);
```

Output:-

```
Original: [ 10, 20, 30, 40, 50 ]
Map: [ 20, 40, 60, 80, 100 ]
Filter: [ 30, 40, 50 ]
Reduce: 150
```

PRACTICAL NO.2(E)

Aim:- Create a program that manipulates objects and displays output.

Code:-

```
let Student={
   name:"Amit",
   age:20,
   course:"BCA",
   marks:78
};

//Add a new property
Student.grade="A";

//Update marks
Student.marks=85;

//Disply students details
console.log(Student);
```

Output:-

```
{ name: 'Amit', age: 20, course: 'BCA', marks: 85, grade: 'A' }
```

PRACTICAL NO.2(F)

Aim:- Implement a small JavaScript program combining function, arrays, and objects.

Code:-

```
//Array of objects
let students=[
    {name:"John",marks:85},
    {name:"Alice",marks:92},
    {name:"Rahul",marks:78}
];

//Function
function displayStudents(studentsList){
    studentsList.forEach(student=>{
        console.log(student.name+"-"+student.marks);
    });
}

//Function Call
displayStudents(students);
```

Output:-

```
John-85
Alice-92
Rahul-78
```

PRACTICAL NO.3(D)

Aim:- Apply basic CSS styling to a React component.

Code:-

App.jsx:-

```
import "./App.css";

function App(){
  return(
    <div className="box">
      <h1>React CSS Example</h1>
      <p>Welcome to MERN Stack Development</p>
    </div>
  );
}

export default App;
```

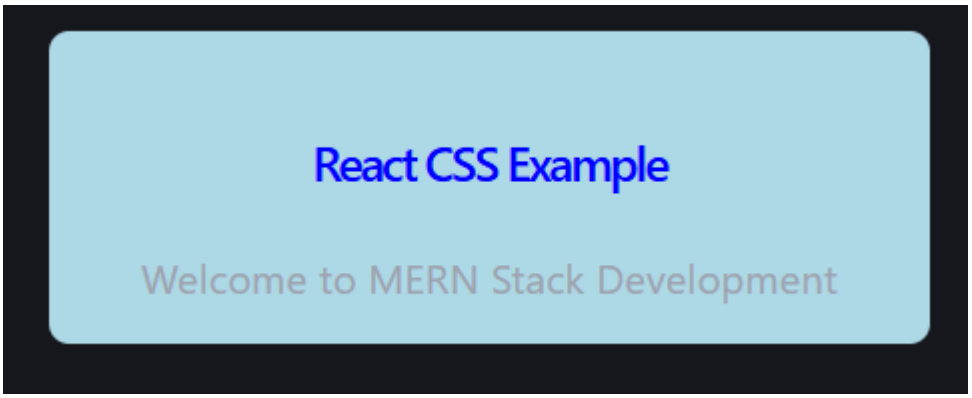
App.css:-

```
.box{
  width: 400px;
  margin: auto;
  margin-top: 50px;
  padding: 20px;
  background-color: lightblue;
  text-align: center;
  border-radius: 10px;
}

h1{
  color: blue;
  font-size: 24px;
}

p{
  font-size: 20px;
}
```

Output:-



PRACTICAL NO.3(E)

Aim:- Create a simple functional component and render it in App.js.

Code:-

App.jsx:-

```
import Welcome from "./components/Welcome";

function App(){
  return(
    <div>
      <Welcome/>
    </div>
  );
}

export default App;
```

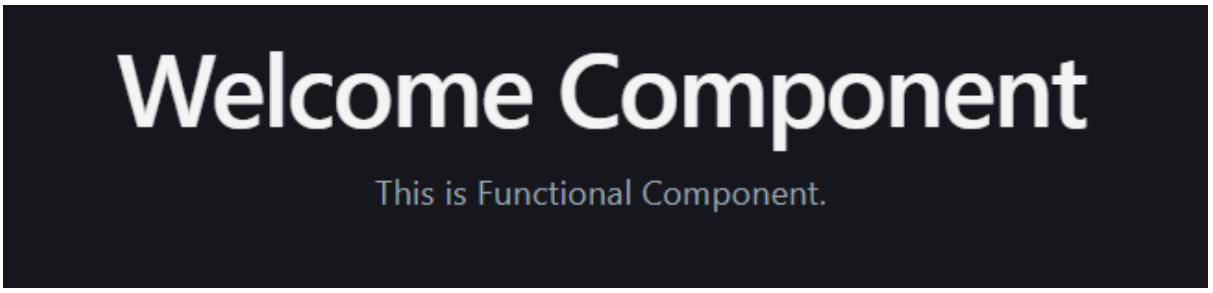
Welcome.jsx:-

```
function Welcome(){
    return(
        <div>
            <h1>Welcome Component</h1>
```

```
            <p>This is Functional Component.</p>
        </div>
    );
}

export default Welcome;
```

Output:-



PRACTICAL NO.3(F)

Aim:- Create a React component that displays the current date and time dynamically using JavaScript.

Create Components folder
Inside src:
Right-click src → New Folder → name it:
-components

Then right-click the components folder:
New File → DateTime.jsx

Code:-

App.jsx:-

```
import DateTime from "./components/DateTime";

function App(){
  return(
    <div>
      <DateTime/>
    </div>
  );
}

export default App;
```

DateTime.jsx:-

```
function DateTime() {

    const current=new Date();

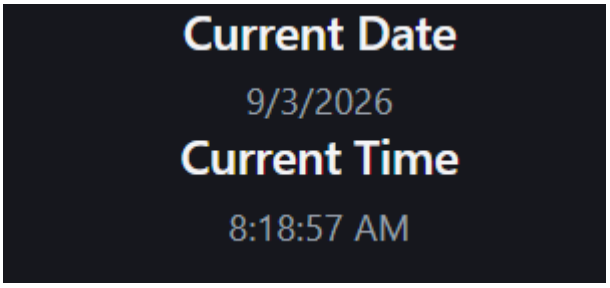
    return(
        <div>
            <h2>Current Date</h2>
            <p>{current.toLocaleDateString()}</p>
```

```
            <h2>Current Time</h2>
            <p>{current.toLocaleTimeString()}</p>
        </div>
    );

}

export default DateTime;
```

Output:-



PRACTICAL 4. React Components, Props, and State

PRACTICAL NO.4(A)

Aim:- Create multiple components and render them on a single page.

Code:-

```
import React from "react";

function Header(){
  return <h1>Welcome to My React App</h1>;
}

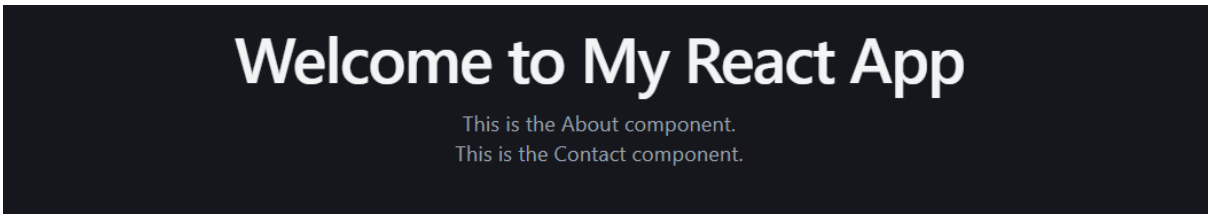
function About(){
  return <p>This is the About component.</p>;
}

function Contact(){
  return <p>This is the Contact component.</p>
}

function App(){
  return(
    <div>
      <Header/>
      <About/>
      <Contact/>
    </div>
  );
}

export default App;
```

Output:-



PRACTICAL NO.4(B)

Aim:- Pass data between components using props.

Code:-

```
import React from "react";

function Student(props){
  return(
    <div>
      <h2>Student Details:</h2>
      <p>Name:{props.name}</p>
      <p>Course:{props.course}</p>
    </div>
  );
}

function App(){
  return(
    <div>
      <h1>Props Example</h1>

      <Student
      name="Rahul"
      course="B.Sc Information Technology"/>

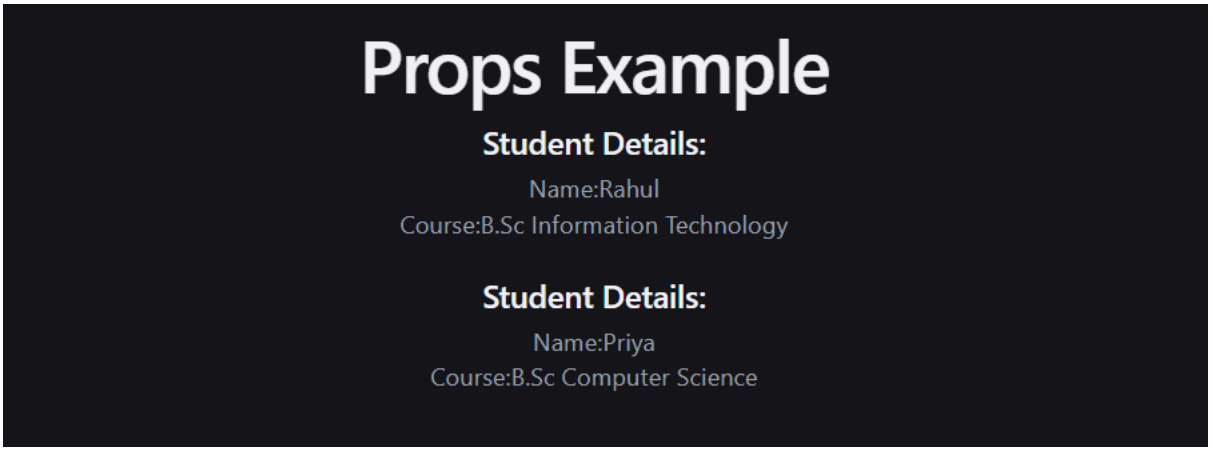
      <br/>

      <Student
```

```
      name="Priya"
      course="B.Sc Computer Science"
      />
    </div>
  );
}

export default App;
```

Output:-



PRACTICAL NO.4(C)

Aim:- Use the useState hook to manage and update data.

Code:-

```
import React,{useState} from "react";

function App(){
  const[name,setName]=useState("Rahul");

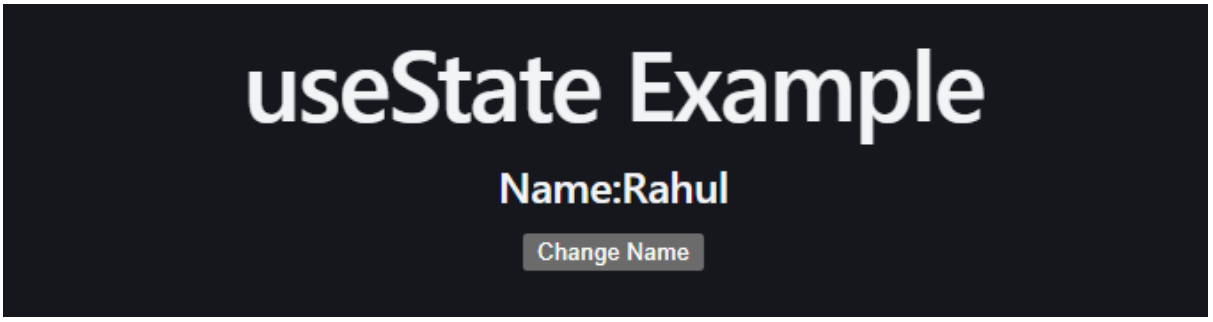
    function changeName(){
      setName("Amit");
    }

    return(
```

```
    <div>
      <h1>useState Example</h1>
      <h2>Name: {name}</h2>
      <button onClick={changeName}>Change Name</button>
    </div>
  );
}

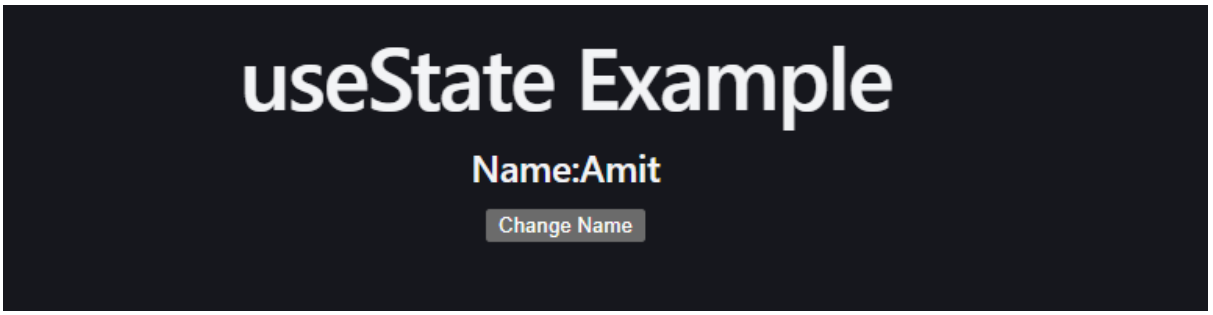
export default App;
```

Initial Output:-



Click on Change Name button → Name changes from **Rahul** to **Amit**.

Final Output:-



PRACTICAL NO.4(D)

Aim:- Create a form component to accept user input and display it dynamically.

Code:-

```
import React,{useState} from "react";

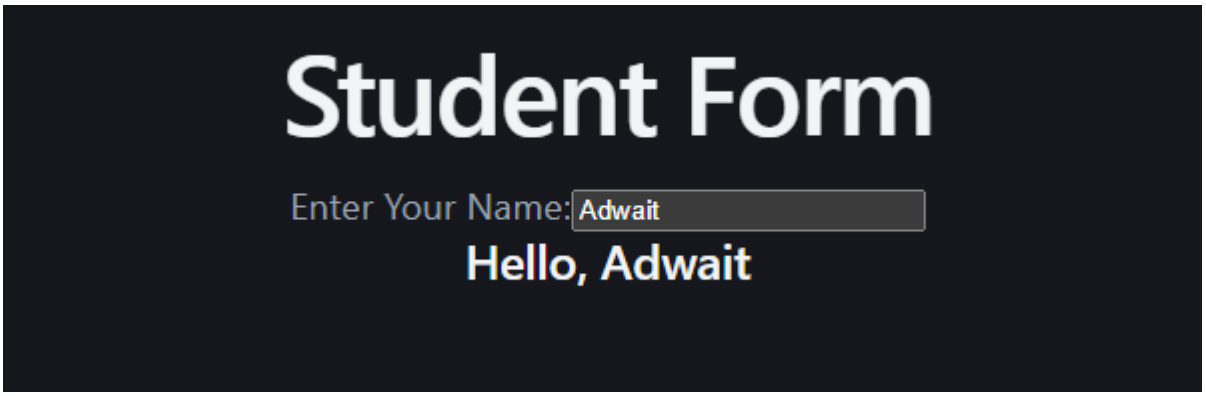
function  StudentForm(){
  const[name,setName]=useState("");

  return(
    <div>
      <h1>Student Form</h1>
      <form>
        <label>Enter Your Name:</label>
        <input
        type="text"
        value={name}
        onChange={(e)=>setName(e.target.value)}
        placeholder="Enter Name"
        />
      </form>
      <h2>Hello, {name}</h2>
    </div>
  );
}

function App(){
  return(
    <StudentForm/>
  );
}

export default App;
```

Output:-



Practical No 8
API Creation and Integration

A. Create a simple REST API using Express

Step 1: Create a project folder

Open VS Code → Terminal:

>mkdir mern-api
>cd mern-api

Step 2: Initialize Node.js
>npm.cmd init -y

Step 3: Install Express
>npm.cmd install express

Step 4: Create server.js

Code:

```
const  express  = require("express");
const app = express();
app.use(express.json());
app.listen(5000,()=>{
   console.log("server is running on 5000");
});
```

Step 5: Run the server
>node server.js

Output:

```
PS C:\Users\student\mern-api> node server.js
server is running on 5000
```

B. Implement a GET API to retrieve data.

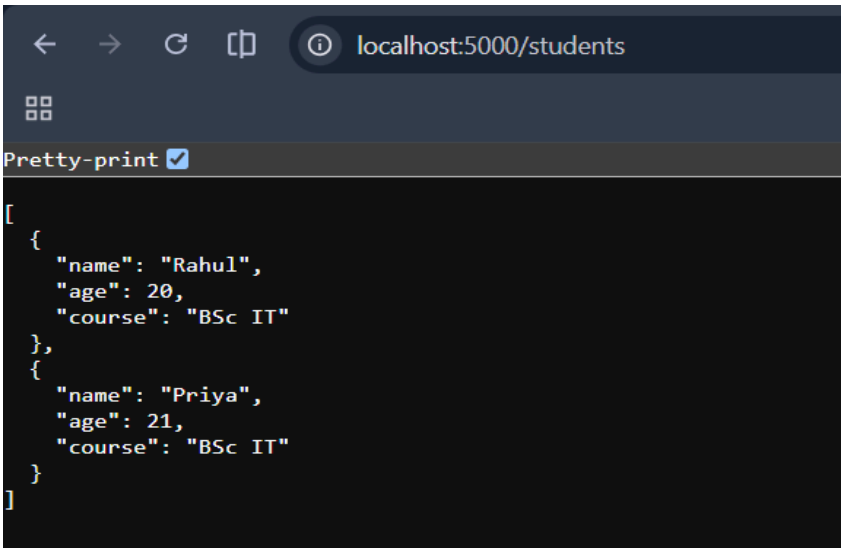
Code:

```
const express = require("express");
const app = express();
app.use(express.json());
app.get("/students", (req, res) => {
   res.json([
      { name: "Rahul", age: 20, course: "BSc IT" },
      { name: "Priya", age: 21, course: "BSc IT" }
   ]);
});
app.listen(5000, () => {
   console.log("Server running on port 5000");
});
```

Run:
>node server.js

Open browser:
http://localhost:5000/students

Output:



C. Fetch data from the API in a React application.

Step 1: Create React application

>npm create vite@latest student-frontend

Select:
React
JavaScript

Then:
>cd student-frontend

Step 2: Install Axios & CORS

In the React project:
>npm install axios

In the mern-api folder:
> npm.cmd install cors

Step 3:

App.jsx

```
import axios from "axios";
import { useEffect, useState } from "react";
function app() {
 const [students, setStudents] = useState([]);

 useEffect(() => {
  axios
   .get("http://localhost:5000/students")
   .then((response) => {
    setStudents(response.data);
   })
   .catch((error) => {
    console.log(error);
   });
 },[]);

 return(
  <div>
   <h1>Student List</h1>
```

```
      {students.map((student,idx)=>(
        <p key={idx}>
          {student.name}-{student.age}-{student.course}
        </p>
      ))}
    </div>
  )
}
export default app;
```

server.js

```
const  express  = require("express");
const cors = require("cors");
const app = express();
app.use(cors());
app.use(express.json());

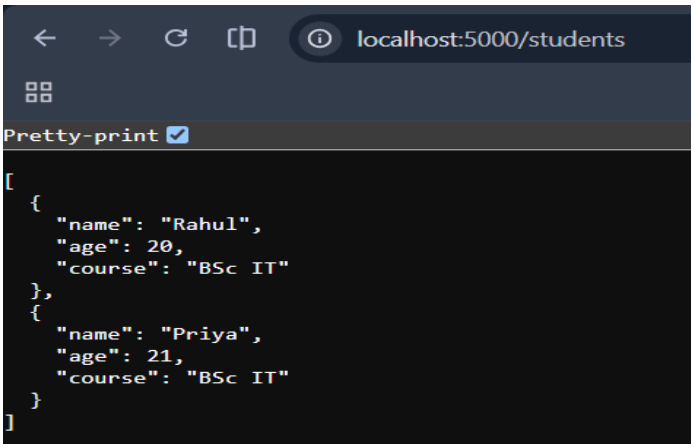
app.get("/students", (req, res) => {
    res.json([
        { name: "Rahul", age: 20, course: "BSc IT" },
        { name: "Priya", age: 21, course: "BSc IT" }
    ]);
});
app.listen(5000,()=>{
    console.log("server is running on 5000");
});
```

Run both projects

Terminal 1 — backend:
>cd mern-api
>node server.js

Output :
Server running on port 5000

Open the URL
http://localhost:5000/students



Terminal 2 — frontend:
>cd student-frontend
>npm run dev

Output:
Open the React URL shown by Vite, usually:

http://localhost:5173/



D. Send data from React to the server using a POST request.

Step 1: Add POST route in server.js

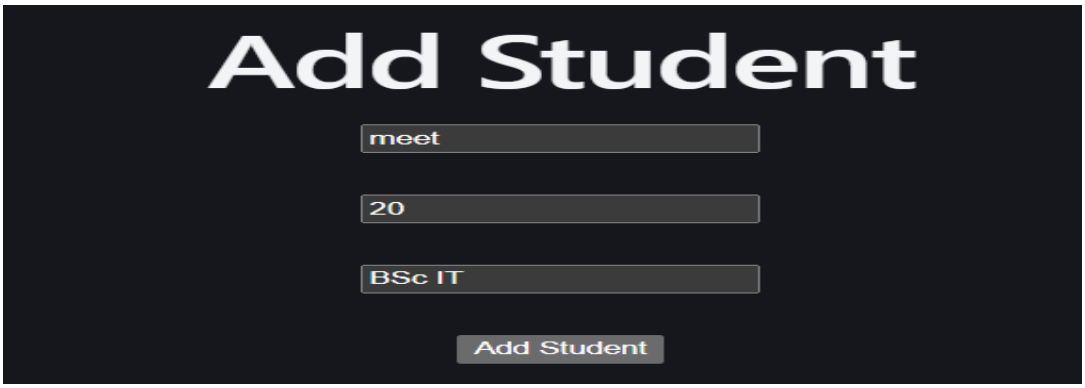
```
app.post("/students", (req, res) => {
    const student = req.body;
    console.log(student);
    res.json({
        message: "Student data received",
        student: student
    });
});
```

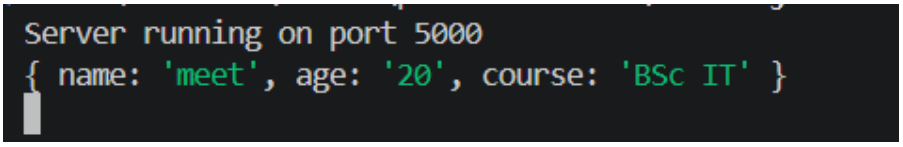
Step 2: Create a form in React

```
import { useState } from "react";
import axios from "axios";
function App() {
  const [name, setName] = useState("");
  const [age, setAge] = useState("");
  const [course, setCourse] = useState("");
  const addStudent = async (e) => {
    e.preventDefault();
    const student = {
      name: name,
      age: age,
      course: course,
    };
    const response = await axios.post(
      "http://localhost:5000/students",
      student,
    );
    console.log(response.data);
  };
  return (
    <div>
      <h1>Add Student</h1>
      <form onSubmit={addStudent}>
        <input
          type="text"
          placeholder="Enter name"
          value={name}
          onChange={(e) => setName(e.target.value)}
        />
        <br />
        <br />
        <input
          type="number"
          placeholder="Enter age"
          value={age}
          onChange={(e) => setAge(e.target.value)}
        />
        <br />
        <br />
        <input
          type="text"
          placeholder="Enter course"
```

```
          value={course}
          onChange={(e) => setCourse(e.target.value)}
        />
        <br />
        <br />
        <button type="submit">Add Student</button>
      </form>
    </div>
  );
}
export default App;
```

Output:





E. Store data in MongoDB.

Step 1: Install Mongoose

In the backend folder:
>npm install mongoose

Step 2: Connect MongoDB

Step 3: Write Code in **sever.js**

```
const express = require("express");
const mongoose = require("mongoose");
const cors= require("cors");
const app = express();
```

```
app.use(cors());

// Middleware
app.use(express.json());


// Connect to MongoDB
mongoose.connect("mongodb://127.0.0.1:27017/studentDB")
    .then(() => {
        console.log("MongoDB connected");
    })
    .catch((error) => {
        console.log("MongoDB connection error:", error);
    });


// Student Schema
const studentSchema = new mongoose.Schema({
    name: String,
    age: Number,
    course: String
});


// Student Model
const Student = mongoose.model("Student", studentSchema);


// POST API - Save student
app.post("/students", async (req, res) => {

    try {

        const student = new Student(req.body);

        await student.save();

        res.json({
            message: "Student saved successfully"
        });

    } catch (error) {

        res.status(500).json({
            message: "Error saving student",
            error: error.message
        });
```

```
    }

});


// Start Server
app.listen(3000, () => {
    console.log("Server running on http://localhost:3000");
});
```

Step 4: write code in App.js

```
import React, { useState } from "react";
import axios from "axios";

function App() {

    const [name, setName] = useState("");
    const [age, setAge] = useState("");
    const [course, setCourse] = useState("");

    const handleSubmit = async (e) => {

        e.preventDefault();

        try {

            const response = await axios.post(
                "http://localhost:3000/students",
                {
                    name: name,
                    age: Number(age),
                    course: course
                }
            );

            alert(response.data.message);

        } catch (error) {

            console.log(error);
            alert("Error saving student");

        }
    };
```

```
  return (
    <div>

      <h1>Student Form</h1>

      <form onSubmit={handleSubmit}>

        <input
          type="text"
          placeholder="Enter Name"
          value={name}
          onChange={(e) => setName(e.target.value)}
        />

        <br /><br />

        <input
          type="number"
          placeholder="Enter Age"
          value={age}
          onChange={(e) => setAge(e.target.value)}
        />

        <br /><br />

        <input
          type="text"
          placeholder="Enter Course"
          value={course}
          onChange={(e) => setCourse(e.target.value)}
        />

        <br /><br />

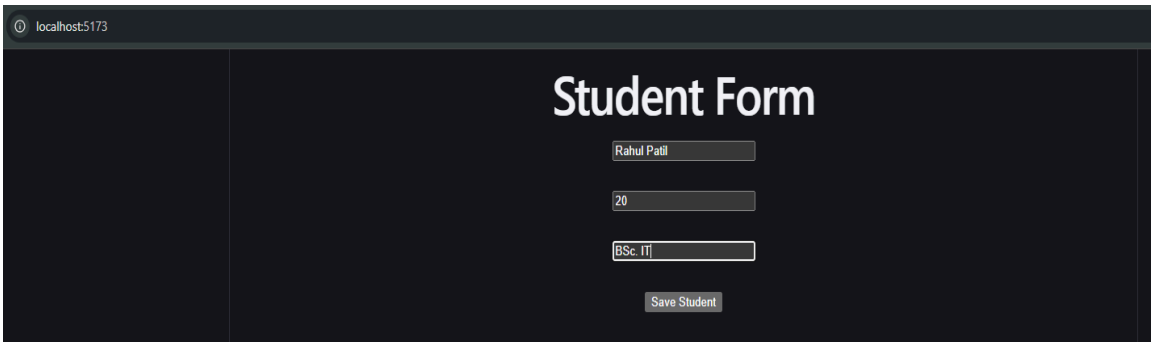
        <button type="submit">
          Save Student
        </button>

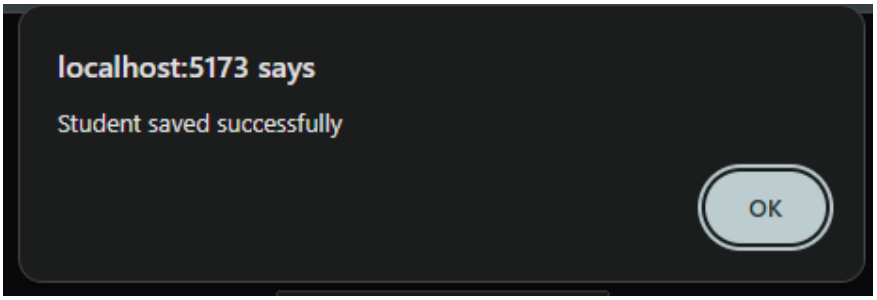
      </form>

    </div>
  );
}

export default App;
```

Output:


localhost:5173
Student Form
Rahul Patil
20
BSc. IT
Save Student


localhost:5173 says
Student saved successfully
OK

F. Display stored data in React page.

Step 1: Create GET API

In server.js:

```
app.get("/students", async (req, res) => {
 const students = await Student.find();
 res.json(students);
});
```

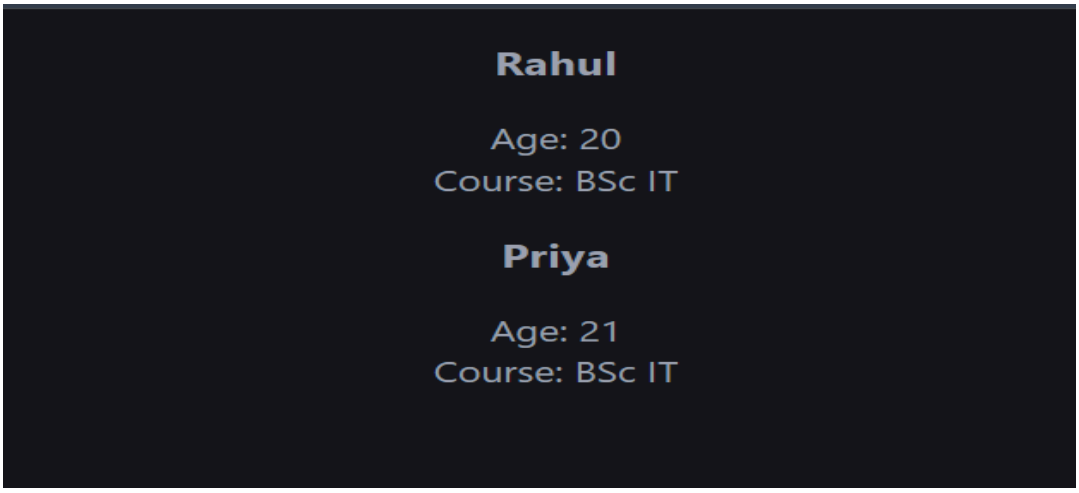
Step 2: React fetches the data

```
useEffect(() => {
   axios.get("http://localhost:5000/students")
         .then((response) => {
     setStudents(response.data);
   });
  }, []);
```

Step 3: Display it

```
{students.map((student) => (
       <div key={student._id}>
         <h3>{student.name}</h3>
         <p>Age: {student.age}</p>
         <p>Course: {student.course}</p>
       </div>
     ))}
```

Output:



Practical No 8
API Creation and Integration

A. Create a simple REST API using Express

Step 1: Create a project folder

Open VS Code → Terminal:

>mkdir mern-api
>cd mern-api

Step 2: Initialize Node.js
>npm.cmd init -y

Step 3: Install Express
>npm.cmd install express

Step 4: Create server.js

Code:

```
const  express  = require("express");
const app = express();
app.use(express.json());
app.listen(5000,()=>{
   console.log("server is running on 5000");
});
```

Step 5: Run the server
>node server.js

Output:

```
PS C:\Users\student\mern-api> node server.js
server is running on 5000
```

B. Implement a GET API to retrieve data.

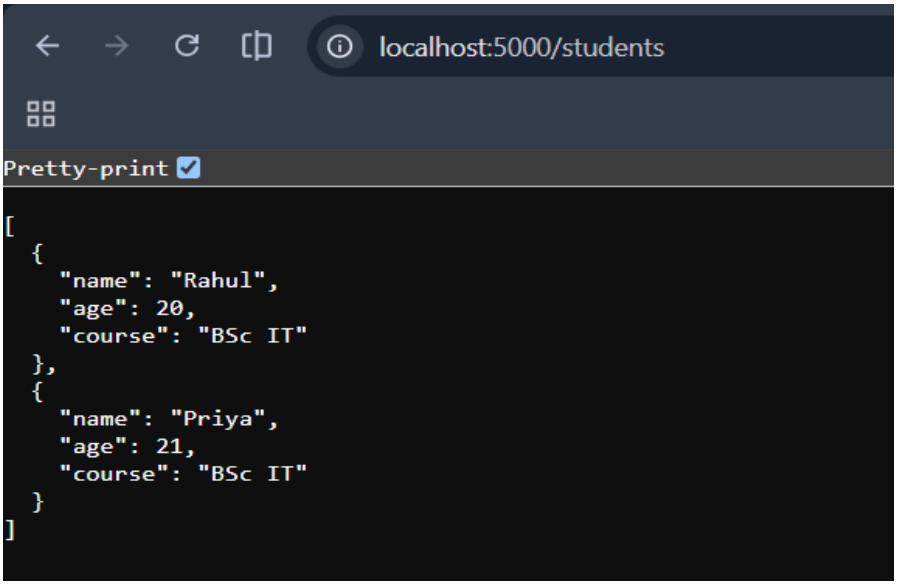
Code:

```
const express = require("express");
const app = express();
app.use(express.json());
app.get("/students", (req, res) => {
   res.json([
      { name: "Rahul", age: 20, course: "BSc IT" },
      { name: "Priya", age: 21, course: "BSc IT" }
   ]);
});
app.listen(5000, () => {
   console.log("Server running on port 5000");
});
```

Run:
>node server.js

Open browser:
http://localhost:5000/students

Output:



C. Fetch data from the API in a React application.

Step 1: Create React application

>npm create vite@latest student-frontend

Select:
React
JavaScript

Then:
>cd student-frontend

Step 2: Install Axios & CORS

In the React project:
>npm install axios

In the mern-api folder:
> npm.cmd install cors

Step 3:

App.jsx

```
import axios from "axios";
import { useEffect, useState } from "react";
function app() {
  const [students, setStudents] = useState([]);

  useEffect(() => {
    axios
      .get("http://localhost:5000/students")
      .then((response) => {
        setStudents(response.data);
      })
      .catch((error) => {
        console.log(error);
      });
  },[]);

  return(
    <div>
      <h1>Student List</h1>
```

```
      {students.map((student,idx)=>(
        <p key={idx}>
          {student.name}-{student.age}-{student.course}
        </p>
      ))}
    </div>
  )
}
export default app;
```

server.js

```
const  express  = require("express");
const cors = require("cors");
const app = express();
app.use(cors());
app.use(express.json());

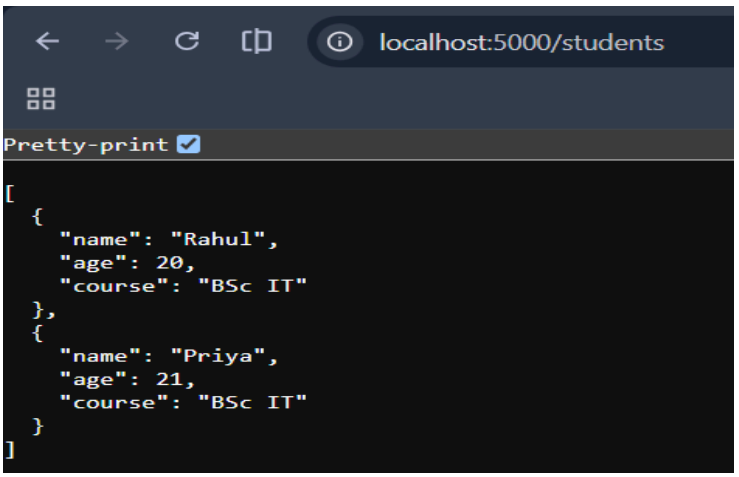
app.get("/students", (req, res) => {
    res.json([
        { name: "Rahul", age: 20, course: "BSc IT" },
        { name: "Priya", age: 21, course: "BSc IT" }
    ]);
});
app.listen(5000,()=>{
    console.log("server is running on 5000");
});
```

Run both projects

Terminal 1 — backend:
>cd mern-api
>node server.js

Output :
Server running on port 5000

Open the URL
http://localhost:5000/students



Terminal 2 — frontend:
>cd student-frontend
>npm run dev

Output:
Open the React URL shown by Vite, usually:

http://localhost:5173/



D. Send data from React to the server using a POST request.

Step 1: Add POST route in server.js

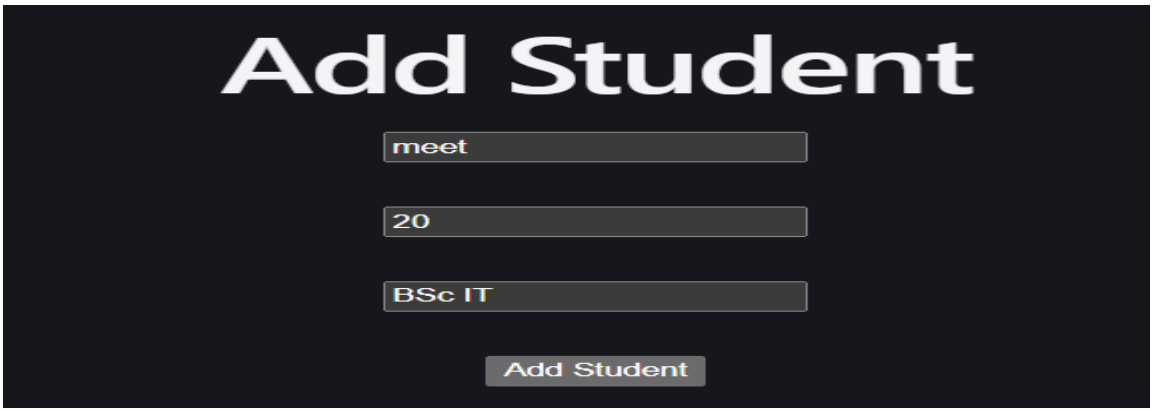
```
app.post("/students", (req, res) => {
    const student = req.body;
    console.log(student);
    res.json({
        message: "Student data received",
        student: student
    });
});
```

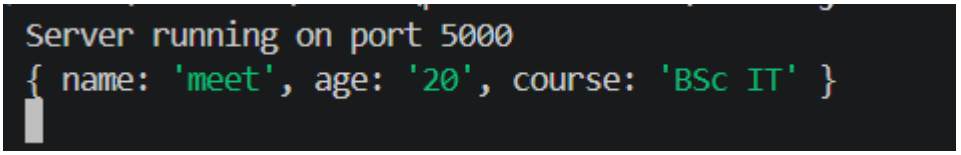
Step 2: Create a form in React

```
import { useState } from "react";
import axios from "axios";
function App() {
  const [name, setName] = useState("");
  const [age, setAge] = useState("");
  const [course, setCourse] = useState("");
  const addStudent = async (e) => {
    e.preventDefault();
    const student = {
      name: name,
      age: age,
      course: course,
    };
    const response = await axios.post(
      "http://localhost:5000/students",
      student,
    );
    console.log(response.data);
  };
  return (
    <div>
      <h1>Add Student</h1>
      <form onSubmit={addStudent}>
        <input
          type="text"
          placeholder="Enter name"
          value={name}
          onChange={(e) => setName(e.target.value)}
        />
        <br />
        <br />
        <input
          type="number"
          placeholder="Enter age"
          value={age}
          onChange={(e) => setAge(e.target.value)}
        />
        <br />
        <br />
        <input
          type="text"
          placeholder="Enter course"
```

```
          value={course}
          onChange={(e) => setCourse(e.target.value)}
        />
        <br />
        <br />
        <button type="submit">Add Student</button>
      </form>
    </div>
  );
}
export default App;
```

Output:





E. Store data in MongoDB.

Step 1: Install Mongoose

In the backend folder:
>npm install mongoose

Step 2: Connect MongoDB

Step 3: Write Code in **sever.js**

```
const express = require("express");
const mongoose = require("mongoose");
const cors= require("cors");
const app = express();
```

```
app.use(cors());

// Middleware
app.use(express.json());


// Connect to MongoDB
mongoose.connect("mongodb://127.0.0.1:27017/studentDB")
    .then(() => {
        console.log("MongoDB connected");
    })
    .catch((error) => {
        console.log("MongoDB connection error:", error);
    });


// Student Schema
const studentSchema = new mongoose.Schema({
    name: String,
    age: Number,
    course: String
});


// Student Model
const Student = mongoose.model("Student", studentSchema);


// POST API - Save student
app.post("/students", async (req, res) => {

    try {

        const student = new Student(req.body);

        await student.save();

        res.json({
            message: "Student saved successfully"
        });

    } catch (error) {

        res.status(500).json({
            message: "Error saving student",
            error: error.message
        });
```

```
    }

});


// Start Server
app.listen(3000, () => {
    console.log("Server running on http://localhost:3000");
});
```

Step 4: write code in App.js

```
import React, { useState } from "react";
import axios from "axios";

function App() {

    const [name, setName] = useState("");
    const [age, setAge] = useState("");
    const [course, setCourse] = useState("");

    const handleSubmit = async (e) => {

        e.preventDefault();

        try {

            const response = await axios.post(
                "http://localhost:3000/students",
                {
                    name: name,
                    age: Number(age),
                    course: course
                }
            );

            alert(response.data.message);

        } catch (error) {

            console.log(error);
            alert("Error saving student");

        }
    };
```

```
  return (
    <div>

      <h1>Student Form</h1>

      <form onSubmit={handleSubmit}>

        <input
          type="text"
          placeholder="Enter Name"
          value={name}
          onChange={(e) => setName(e.target.value)}
        />

        <br /><br />

        <input
          type="number"
          placeholder="Enter Age"
          value={age}
          onChange={(e) => setAge(e.target.value)}
        />

        <br /><br />

        <input
          type="text"
          placeholder="Enter Course"
          value={course}
          onChange={(e) => setCourse(e.target.value)}
        />

        <br /><br />

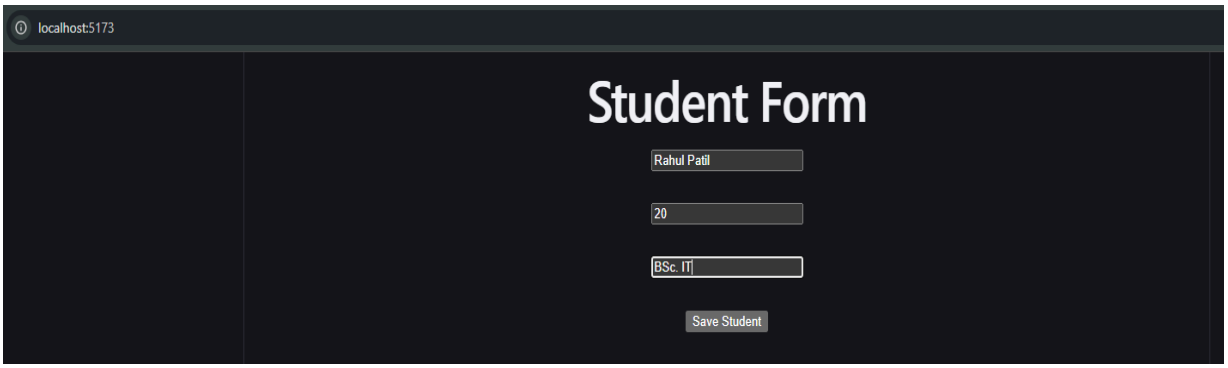
        <button type="submit">
          Save Student
        </button>

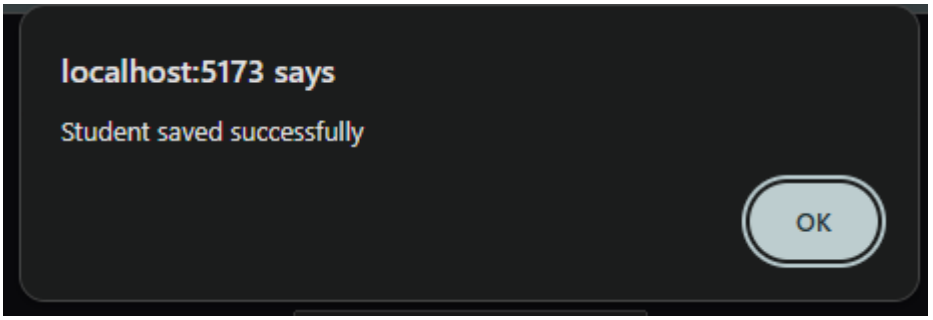
      </form>

    </div>
  );
}

export default App;
```

Output:


localhost:5173
Student Form
Rahul Patil
20
BSc. IT
Save Student


localhost:5173 says
Student saved successfully
OK

F. Display stored data in React page.

Step 1: Create GET API

In server.js:

```
app.get("/students", async (req, res) => {
 const students = await Student.find();
 res.json(students);
});
```

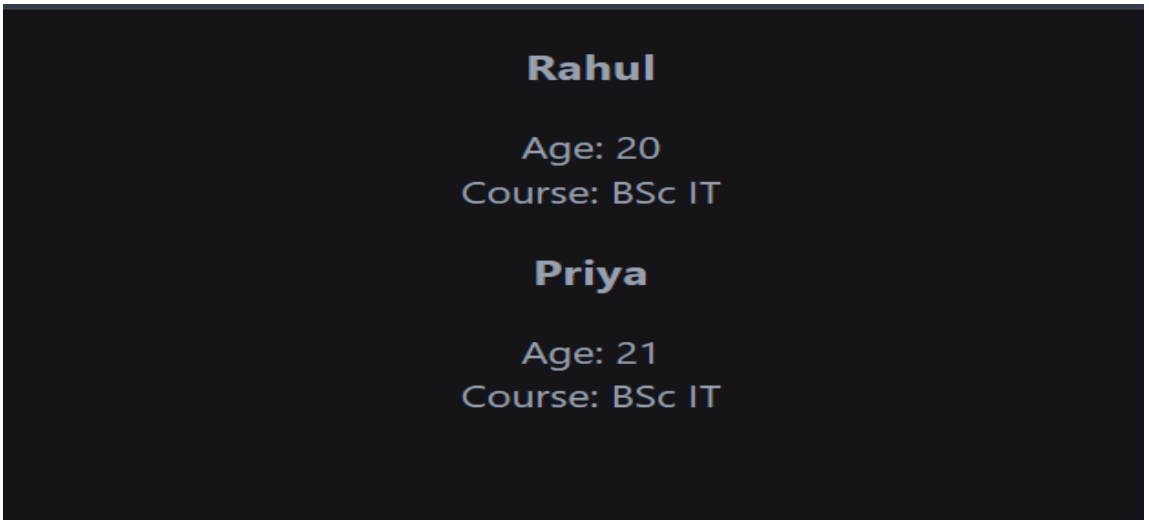
Step 2: React fetches the data

```
useEffect(() => {
    axios.get("http://localhost:5000/students")
          .then((response) => {
      setStudents(response.data);
    });
  }, []);
```

Step 3: Display it

```
{students.map((student) => (
       <div key={student._id}>
         <h3>{student.name}</h3>
         <p>Age: {student.age}</p>
         <p>Course: {student.course}</p>
       </div>
     ))}
```

Output:



Full-Stack Development

PRACTICAL NO. 1(A)

Aim:-Install Node.js and visual Studio code ,and verify installation.

Step1: Install node.js
1.Visit the official Node.js website:
Node.js Download Page.
2.Download the LTS(Long Term Support) version.
3.Run the installer and Follow the installation wizard.
4.Keep all default setting and click Next until installation completes.\

Step 2: Install visual studio code:
1.Visit: Visual Studio Code
2.Download and Install VS Code.
3.Open VS Code after installation.

Step 3: Verify installation:
Open Command Prompt Or Terminal and type:
Node -v

Output:
Npm -v

Output:
10.5.0

PRACTICAL NO. 1(B)

Aim:- Create a simple Node.js File and run it.

Step 1: Create a Folder
Create a folder named:
NodeJS_practical

Step 2: Open Folder in VS Code
1.Open Vs Code
2.Select FIle >> Open Folder
3.Choose NOdeJS_practical

Step 3: Create a File
Create a file named:
app.js

Step 4: Write Code
console.log("Hello world");

Step 5: Run the Program
Open Terminal in VS Code:
Node app.js

Output:
Hello world

PRACTICAL NO. 1(C)

Aim:- Write a program demonstrating basic javascript command.

Code:-

```
console.log("Welcome to node js");
let name="John";
let age=20;
console.log("Name:",name);
console.log("Age:",age);
let num1=15;
let num2=5;
console.log("First Number:",num1);
console.log("Second Number:",num2);
console.log("Addition:",num1+num2);
console.log("Substraction:",num1-num2);
console.log("Multiplication:",num1*num2);
console.log("Division:",num1/num2);
console.log("Modulus:",num1%num2);
```

Output:-

```
Welcome to node js
Name: John
Age: 20
First Number: 15
Second Number: 5
Addition: 20
Substraction: 10
Multiplication: 75
Division: 3
Modulus: 0
```

PRACTICAL NO. 1(D)

Aim:- Create and run a simple program using variable and functions.

Code:-

```
let name= "Adwait";
let marks=85;
function displayStudent(studentName, studentMarks)
{
console.log("Student Name:",studentName);
console.log("Marks:",studentMarks);
}
displayStudent(name, marks);
```

Output:-

Student Name: Adwait
Marks: 85

(Sum of Two Numbers)

Code:-

```
let num1=10;
let num2=20;

function add(a,b){
   return a+b;
}

let result=add(num1,num2);

console.log("First Number:",num1);
console.log("Second Number:",num2);
console.log("Sum:",result);
```

Output:-

First Number: 10
Second Number: 20
Sum: 30

PRACTICAL NO. 1(E)

Aim:- Demonstrate the use of operator and control statement in javascript.

(Operators)

Code:-

```
let a=15;
let b=5;

//Arithmetic Operator
console.log("Addition:",a+b);
console.log("Substraction:",a-b);
console.log("Multiplication:",a*b);
console.log("Divison:",a/b);
console.log("Modulus:",a%b);

//Comparision Operator
console.log("Is a greater than b?",a>b);

//Logical Operator
console.log("a>10 && b<10",a>10 && b<10);
```

Output:-

Addition: 20
Substraction: 10
Multiplication: 75
Divison: 3
Modulus: 0
Is a greater than b? true
a>10 && b<10 true

(If - Else Statement)

Code:-

```
let marks=80;
if(marks>=50){
   console.log("Result:Pass");
}else{
   console.log("Result:Fail");
}
```

Output:-

Result: Pass

(Switch Case)

Code:-

```
let day=3;

switch(day){
   case 1:
      console.log("Monday");
      break;
   case 2:
      console.log("Tuesday");
      break;
   case 3:
      console.log("Wednesday");
      break;
   case 4:
      console.log("Thursday");
      break;
   default:
      console.log("Invalid Day!");
}
```

Output:-

Wednesday

PRACTICAL NO. 1(F)

Aim:- Write a program using loops and template literals.

(For Loop)

Code:-

```
let student="Amit";
console.log(`Welcome,${student}!`);

for(let i=1;i<=5;i++)
{
   console.log(`Iteration ${i}:${student} is learning Node.js`);
}
```

Output:-

Iteration 1:Amit is learning Node.js

Iteration 2:Amit is learning Node.js

Iteration 3:Amit is learning Node.js

Iteration 4:Amit is learning Node.js

Iteration 5:Amit is learning Node.js

(While Loop)

Code:-

```
let number=1;

while(number<=5)
{
   console.log(`The square of ${number} is ${number*number}`);
   number++;
}
```

Output:-

The square of 1 is 1
The square of 2 is 4
The square of 3 is 9
The square of 4 is 16
The square of 5 is 25

(Do-While Loop)

Code:-

```
let count=1;

do{
   console.log(`Count value is ${count}`);
   count++;
} while(count<=5);
```

Output:-

```
Count value is 1
Count value is 2
Count value is 3
Count value is 4
Count value is 5
```

(Multiplication Table)

Code:-

```
let num=7;

console.log(`Multiplication Table of ${num}`);

for(let i=1;i<=10;i++)
{
   console.log(`${num}x${i}=${num*i}`);
}
```

Output:-

```
7x1=7
7x2=14
7x3=21
7x4=28
7x5=35
7x6=42
7x7=49
7x8=56
7x9=63
7x10=70
```

(Sum of 10 Numbers)

Code:-

```
let sum=0;

for(let i=1;i<=10;i++)
{
   sum+=i;
}
console.log(`The sum of numbers from 1 to 10 is ${sum}.`);
```

Output:-

The sum of numbers from 1 to 10 is 55.

(Eligibility Criteria for Voting)

Code:-

```
let age=20;

if(age>=18){
   console.log("Person is eligible to vote.");
}
else{
   console.log("Person is not eligible to vote.")
}
```

Output:-

Person is eligible to vote.

(Grade Checker Using Switch Case)

Code:-

```
let marks = 85;

switch (true) {
   case (marks>=90 && marks<=100):
      console.log("Grade: A+");
      break;

   case (marks>=80):
      console.log("Grade: A");
      break;

   case (marks>=70):
      console.log("Grade: B");
      break;

   case (marks>=60):
      console.log("Grade: C");
      break;

   case (marks>=50):
      console.log("Grade: D");
      break;

   case (marks>=0):
      console.log("Grade: F");
      break;

   default:
      console.log("Invalid marks");
}
```

Output:-

Grade: A

PRACTICAL 5. React Routing and Data Handling

PRACTICAL NO.5(A)

Aim:- Install and configure React Router in the React application.

Procedure:-

Step 1: Create a New React Application

Open **Command Prompt (CMD)** and run the following command:

→ npm create vite@latest react-routing-app -- --template react

This command creates a new React application named **react-routing-app** using Vite and the React template.

After executing the command, Vite asks:

Which linter to use?

Select → ESLint

Then it asks:

Install with npm and start now?

Select → Yes

Vite then creates the project and installs the required dependencies. After successful installation, the development server starts automatically.

The CMD displays:

VITE v8.2.2 ready in 241 ms

➜ Local: http://localhost:5173/
➜ Network: use --host to expose
➜ press h + enter to show help

This indicates that the React application has been successfully created and is running on the local development server.

Step 2: Open the React Application on Localhost

Open a web browser and visit:

http://localhost:5173/

The default Vite + React application will be displayed.

At this stage, the application is running continuously in the CMD.

Step 3: Stop the Running Development Server

Before installing/configuring additional packages from the same CMD, the currently running Vite development server can be terminated.

Press → Ctrl + C

The CMD may display:

Terminate batch job (Y/N)?

Type → Y

and press **Enter**.

The Vite development server will stop and the normal CMD prompt will appear again.

Step 4: Install React Router DOM

Now install the React Router package using:

npm install react-router-dom

The package will be downloaded and added to the project dependencies.

A successful installation displays information similar to:

added 7 packages in 1s

Step 5: Navigate to the React Project Folder

If the CMD is not already inside the project folder, navigate to the project directory using:

cd react-routing-app

The prompt should now look similar to:

C:\Users\user\react-routing-app>

Step 6: Verify React Router Installation

To check whether react-router-dom has been installed successfully, run:

npm list react-router-dom

The output shows the installed version, for example:

react-routing-app@0.0.0 C:\Users\user\react-routing-app
└── react-router-dom@7.18.3

This confirms that **React Router DOM has been successfully installed** in the React application.

Step 7: Start the React Development Server Again

After installing React Router DOM, start the development server using:

npm run dev

Vite starts the development server and displays:

VITE v8.2.2 ready in 155 ms

➜ Local: http://localhost:5173/
➜ Network: use --host to expose
➜ press h + enter to show help

The React application is now running again on:

http://localhost:5173/

Open this address in a browser to verify that the application is working correctly.

Code:-

```
import {BrowserRouter} from "react-router-dom";

function App(){
  return(
    <BrowserRouter>
      <h1>React Router Setup</h1>
    </BrowserRouter>
  );
}
export default App;
```

Output:-



PRACTICAL NO.5(B)

Aim:- Create multiple pages and implement navigation between them.

Code:-

Home.jsx:-

```
function Home() {
   return<h1>Home Page</h1>;
}

export default Home;
```

About.jsx:-

```
function About(){
   return <h1>About Page</h1>;
}

export default About;
```

Contact.jsx:-

```
function Contact(){
   return<h1>Contact</h1>;
}

export default Contact;
```

App.jsx:-

```
import {BrowserRouter,Routes,Route,Link} from "react-router-dom";

import Home from "./Home";
import About from "./About";
import Contact from "./Contact";

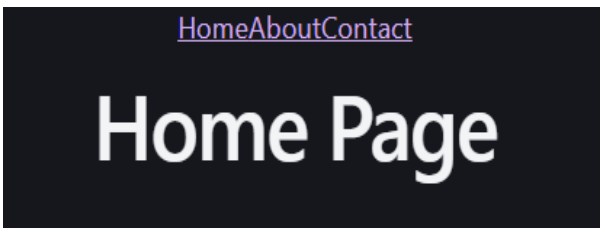
function App() {
  return(
    <BrowserRouter>
      <nav>
        <Link to="/Home">Home</Link>
        <Link to="/About">About</Link>
        <Link to="/Contact">Contact</Link>
      </nav>

      <Routes>
        <Route path="/Home" element={<Home/>}/>
        <Route path="/About" element={<About/>}/>
        <Route path="/Contact" element={<Contact/>}/>
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

Output:-









PRACTICAL NO.5(C)

Aim:- Use Link and Routes to move between pages.

Code:-

```
import {BrowserRouter,Routes,Route,Link} from "react-router-dom";

function Home(){
  return <h2>Home Page</h2>
}

function About() {
  return <h2>About Page</h2>
}

function Services() {
  return <h2>Services Page</h2>
}

function App() {
  return(
    <BrowserRouter>
      <h1>React Navigation</h1>

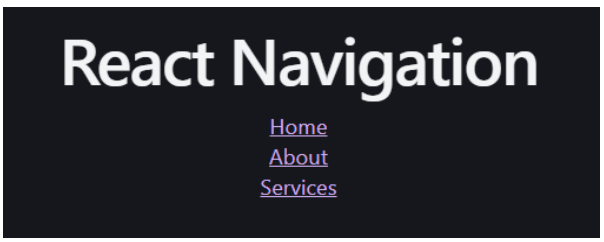
      <Link to="/Home">Home</Link>
      <Link to="/About">About</Link>
      <Link to="/Services">Services</Link>

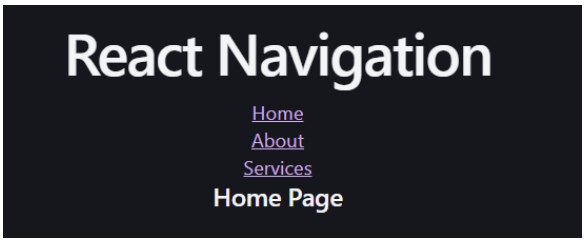
      <Routes>
      <Route path="/Home" element={<Home/>}/>
      <Route path="/About" element={<About/>}/>
      <Route path="/Services" element={<Services/>}/>
      </Routes>

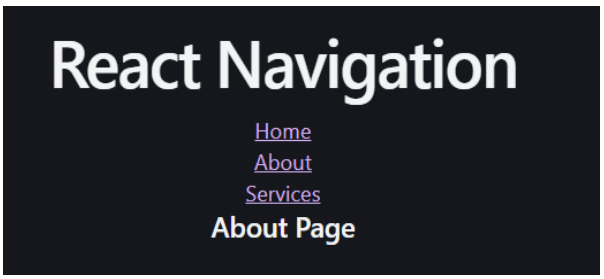
    </BrowserRouter>
  );
}

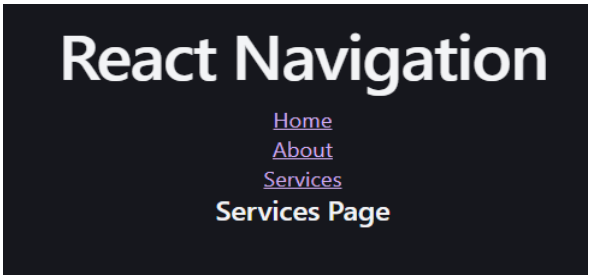
export default App;
```

Output:-









PRACTICAL NO.5(D)

Aim:- Fetch sample data from a public API using Axios.

Code:-

(Install npm axios from new cmd(steps & ss))

```
import {useEffect, useState} from 'react';
import axios from 'axios';

function App() {
  const [user, setUser] = useState([]);

  useEffect(() => {

    axios.get('https://jsonplaceholder.typicode.com/users')

      .then(response => {
        setUser(response.data);
      })

      .catch(error => {
```

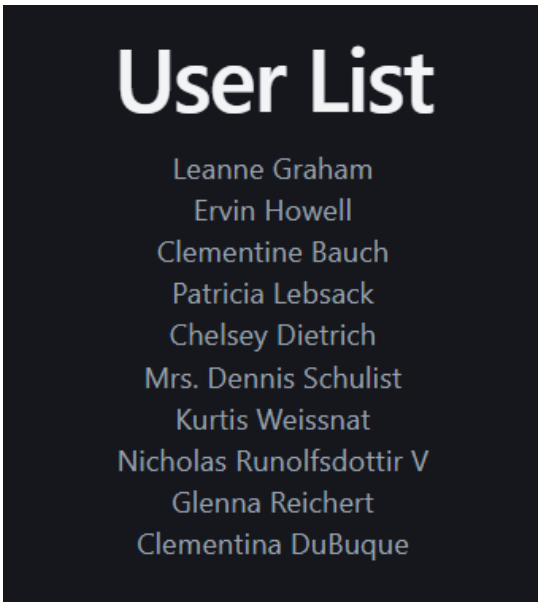
```
        console.error('Error fetching user data:', error);
      });
  }, []);

  return (
    <div>
      <h1>User List</h1>

      {user.map(user => (
        <p key={user.id}>{user.name}</p>
      ))}
    </div>
  );
}

export default App;
```

Output:-



PRACTICAL NO.5(E)

Aim:- Display fetched data in table or list format.

Code:-

```
import {useEffect, useState} from 'react';
import axios from 'axios';

function App() {
  const [user, setUser] = useState([]);

  useEffect(() => {
    axios.get('https://jsonplaceholder.typicode.com/users')
      .then(response => {
        setUser(response.data);
      })
      .catch(error => {
        console.error('Error fetching user data:', error);
      });
  }, []);

  return (
    <div>
      <h1>User List</h1>
      <table border="1" cellPadding="10">
        <thead>
          <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Email</th>
            <th>City</th>
          </tr>
        </thead>
        <tbody>
          {user.map((user) => (
            <tr key={user.id}>
              <td>{user.id}</td>
              <td>{user.name}</td>
              <td>{user.email}</td>
              <td>{user.address.city}</td>
            </tr>
          ))}
```

```
        </tbody>
      </table>
    </div>
  );
}

export default App;
```

Output:-



User List

ID	Name	Email	City
1	Leanne Graham	Sincere@april.biz	Gwenborough
2	Ervin Howell	Shanna@melissa.tv	Wisokyburgh
3	Clementine Bauch	Nathan@yesenia.net	McKenziehaven
4	Patricia Lebsack	Julianne.OConner@kory.org	South Elvis
5	Chelsey Dietrich	Lucio_Hettinger@annie.ca	Roscoeview
6	Mrs. Dennis Schulist	Karley_Dach@jasper.info	South Christy
7	Kurtis Weissnat	Telly.Hoeger@billy.biz	Howemouth
8	Nicholas Runolfsdottir V	Sherwood@rosamond.me	Aliyaview
9	Glenna Reichert	Chaim_McDermott@dana.io	Bartholomebury
10	Clementina DuBuque	Rey.Padberg@karina.biz	Lebsackbury

PRACTICAL NO.5(F)

Aim:- Implement search or filter functionality on displayed data.

Code:-

```
import { useEffect, useState } from "react";
import axios from "axios";

function App() {
  const [users, setUsers] = useState([]);
  const [search, setSearch] = useState("");

  useEffect(() => {
    axios
      .get("https://jsonplaceholder.typicode.com/users")
      .then((response) => {
        setUsers(response.data);
      })
      .catch((error) => {
        console.error("Error fetching users:", error);
      });
  }, []);

  const filteredUsers = users.filter((user) =>
    user.name.toLowerCase().includes(search.toLowerCase())
  );

  return (
    <div>
      <h1>User Search</h1>

      <input
        type="text"
        placeholder="Search by name"
        value={search}
        onChange={(e) => setSearch(e.target.value)}
      />

      <br />
      <br />

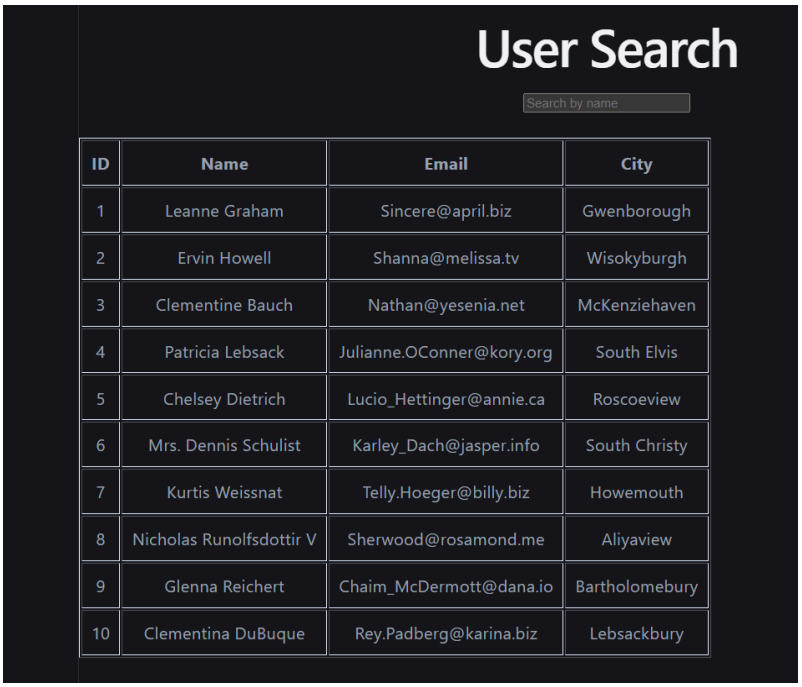
      <table border="1" cellPadding="10">
```

```
        <thead>
          <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Email</th>
            <th>City</th>
          </tr>
        </thead>

        <tbody>
          {filteredUsers.map((user) => (
            <tr key={user.id}>
              <td>{user.id}</td>
              <td>{user.name}</td>
              <td>{user.email}</td>
              <td>{user.address.city}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

export default App;
```

Output:



PRACTICAL 6. Node.js and Express Fundamentals

PRACTICAL NO.6(A)

Aim:- Install Express and create a simple server.

Procedure:-

Part 1:- Create a New Express Project Using Command Prompt

Step 1:- Open Command Prompt

Open Command Prompt on the computer.

Step 2:- Create a new project folder

Enter the following command:-

mkdir express-practical1

Step 3:- Navigate to the project folder

cd express-practical1

Step 4:- Initialize the Node.js project

npm.cmd init -y

This creates a **package.json** file containing the basic configuration of the Node.js project.

Step 5: Install Express.js

npm.cmd install express

Express.js and its required dependencies will be installed in the project.

After successful installation, the following folders/files are created:

- **node_modules –** Contains installed packages.

- **package-lock.json –** Stores the exact versions of installed packages.
- **package.json –** Contains project information and dependencies.

Part 2: Create **server.js** File Using Visual Studio Code

Step 1:- Open Visual Studio Code

Launch Visual Studio Code.

Step 2:- Import/Open the project folder

1. Click **File** in the top menu.
2. Select **Open Folder...**
3. Navigate to:

C:\Users\user\express-practical1

4. Select the **express-practical1** folder.
5. Click **Select Folder**.

The project folder will now appear in the **Explorer** panel on the left side.

Step 3:- Create a new JavaScript file

1. In the Explorer panel, right-click the **express-practical1** folder.
2. Select **New File**.
3. Enter the file name → server.js
4. Press **Enter**.

The **server.js** file will be created inside the project folder.

Step 4:- Write the Express server code

Open **server.js** and write the following code:-

```
const express=require("express");

const app=express();

const PORT=3000;
```

```
app.get("/",(req,res)=>{
   res.send("Hello from Express!");
});

app.listen(PORT,()=>{
   console.log(`Server is running on
      http://localhost:${PORT}`);
});
```

Run the command (In CMD):-

→ node server.js

Open Browser and visit:-

→ http://localhost:3000

Browser Output:-

Hello from Express!

PRACTICAL NO.6(B)

Aim:- Display a message on the browser using the server response.

Code:-

```
const express=require("express");

const app=express();

const PORT=3000;

app.get("/",(req,res)=>{
   res.send("Hello! Welcome to my Express Server.");
});
```

```
app.listen(PORT,()=>{
   console.log(`Server is running on
      http://localhost:${PORT}`);
});
```

Open Browser and visit:-

→ http://localhost:3000

Browser Output:-

Hello! Welcome to my Express Server.

PRACTICAL NO.6(C)

Aim:- Create different routes in Express.

Code:-

```
const express=require("express");

const app=express();

const PORT=3000;

//Home Route
app.get("/",(req,res)=>{
   res.send("Welcome to the Home Page.");
});

//About Route
app.get("/about",(req,res)=>{
   res.send("This is About Page");
});

//Contact Route
app.get("/Contact",(req,res)=>{
   res.send("This is Contact Page");
});
```

```
//Products Route
app.get("/Products",(req,res)=>{
   res.send("This is Products Page");
});

app.listen(PORT,()=>{
   console.log(`Server is running on
      http://localhost:${PORT}`);
});
```

Open Browser and visit:-

→ http://localhost:3000

Browser Output:-

→ Visit http://localhost:3000

Welcome to the Home Page.

→ Visit http://localhost:3000/About

This is About Page

→ Visit http://localhost:3000/Contact

This is Contact Page

→ Visit http://localhost:300/Products

This is Products Page

PRACTICAL NO.6(D)

Aim:- Send a JSON response from the server.

Code:-

```
const express=require("express");

const app=express();

const PORT=3000;

//Home Route
app.get("/",(req,res)=>{
   res.send("Welcome to the Home Page.");
});

//About Route
app.get("/about",(req,res)=>{
   res.send("This is About Page");
});

//Contact Route
app.get("/Contact",(req,res)=>{
   res.send("This is Contact Page");
});

//Products Route
app.get("/Products",(req,res)=>{
   res.send("This is Products Page");
});

app.get("/api/user",(req,res)=>{
   res.json({
      id:1,
      name:"John Doe",
      email: "john@example.com",
      city: "Mumbai"
   });
});

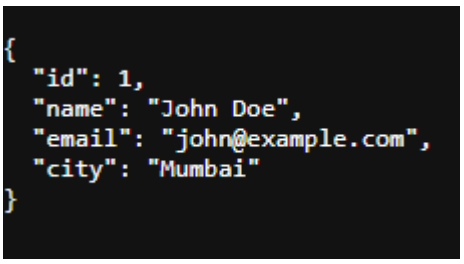
app.listen(PORT,()=>{
   console.log(`Server is running on
```

```
    http://localhost:${PORT}`);
});
```

Open Browser and Visit:-

→ http://localhost:3000/api/user

Browser Output:-



PRACTICAL NO.6(E)

Aim:- Test routes using browser or Postman.

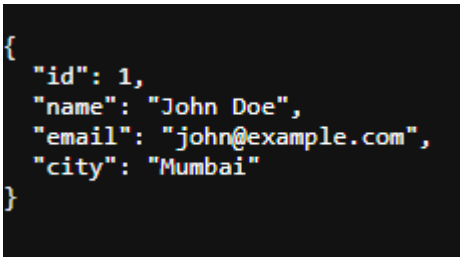
Code:-

In practical 6(c) we create, in practical 6(e) we test.

Route	Method	Expected Output
/	GET	Home Page
/about	GET	About Page
/contact	GET	Contact Page
/products	GET	Products Page
/api/user	GET	JSON user data

Routes directly in the browser.
For Eg:-
→ https://localhost:3000/api/user

Browser Output:-



PRACTICAL NO.6(F)

Aim:- Run the server and observe the output in the console.

Code:-

```
const express=require("express");

const app=express();

const PORT=3000;

//Home Route
app.get("/",(req,res)=>{
   console.log("Home Route accessed");
   res.send("Welcome to the Home Page.");
});

//About Route
app.get("/about",(req,res)=>{
   console.log("About route accessed");
   res.send("This is About Page");
});

//Contact Route
app.get("/Contact",(req,res)=>{
   console.log("Contact route accessed");
```

```
    res.send("This is Contact Page");
});

//Products Route
app.get("/Products",(req,res)=>{
    console.log("Products route accessed");
    res.send("This is Products Page");
});

app.get("/api/user",(req,res)=>{
    console.log("User API accessed");

    res.json({
        id:1,
        name:"John Doe",
        email: "john@example.com",
        city: "Mumbai"
    });
});

app.listen(PORT,()=>{
    console.log(`Server is running on
        http://localhost:${PORT}`);
});
```

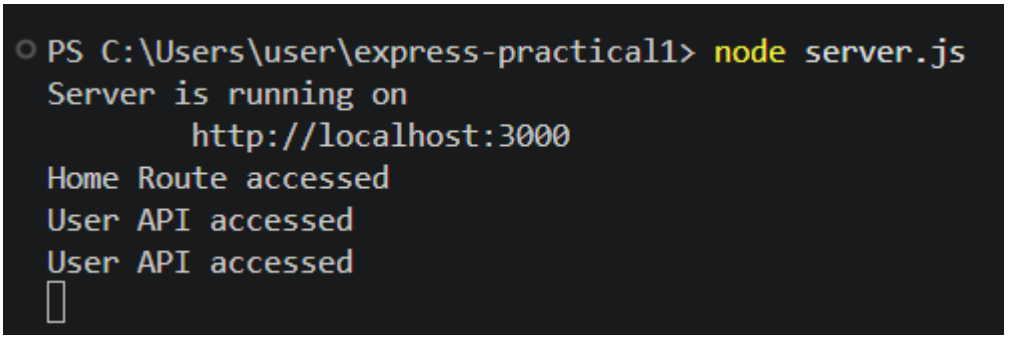
Open terminal in VS Code & enter:-

→ node server.js

Open Browser and visit:-

→ http://localhost:3000
→ http://localhost:3000/api/user

VS Code Output:-



PRACTICAL 7. MongoDB Basics

PRACTICAL NO.7(A)

Aim:- Install and start MongoDB.

Requirements:-

- Windows PC
- Command Prompt (CMD)
- MongoDB Server 4.4
- MongoSH

1. Install MongoDB Server

Step 1: Download MongoDB

Download the **MongoDB Community Server 4.4** installer for Windows.

Run the downloaded **.msi** file.

Step 2: Install MongoDB

1. Open the MongoDB .msi installer.
2. Click **Next**.
3. Accept the license agreement and click **Next**.
4. Select **Complete** installation.
5. Keep the default installation location:

C:\Program Files\MongoDB\Server\4.4\

6. Click **Next** and then **Install**.
7. After installation is completed, click **Finish**.

The **MongoDB executable files** will be available in:

C:\Program Files\MongoDB\Server\4.4\bin

The main server program is:

mongod.exe

2. Install MongoSH

MongoSH is the MongoDB command-line shell used to connect to the MongoDB server.

Step 1: Download MongoSH

Download the MongoSH Windows ZIP package.
After downloading, extract it to:
D:\Fullstack

The **MongoSH** folder should be:
D:\Fullstack\mongosh-2.12.0-win32-x64\mongosh-2.12.0-win32-x64\

The executable files will be inside:
D:\Fullstack\mongosh-2.12.0-win32-x64\mongosh-2.12.0-win32-x64\bin

The main shell program is:
mongosh.exe

3. Create MongoDB Data Directory

Open **Command Prompt as Administrator** and run:

mkdir C:\data
mkdir C:\data\db

MongoDB will use this directory to store database files.

4. Set MongoDB Server Path

In CMD, run:
setx PATH "%PATH%;C:\Program Files\MongoDB\Server\4.4\bin"

5. Set MongoSH Path

Run:

setx PATH

"%PATH%;D:\Fullstack\mongosh-2.12.0-win32-x64\mongosh-2.12.0-win32-x64\bin"

Close CMD and open a **new Command Prompt**.

6. Check MongoDB Installation

Run:

mongod --version

If MongoDB is installed correctly, the MongoDB server version will be displayed.

7. Check MongoSH Installation

Run:

mongosh --version

If MongoSH is installed correctly, the MongoSH version will be displayed.

8. Start MongoDB Server

Run:

mongod --dbpath C:\data\db

MongoDB will start and listen on the default port:

27017

Keep this CMD window open.

9. Start MongoSH

Open a **second Command Prompt** window and type:

mongosh

MongoSH will connect to the MongoDB server.

You should see a connection similar to:
mongodb://127.0.0.1:27017/

This means MongoDB is running successfully.

10. Test MongoDB

Inside MongoSH, run:
show dbs

Create a database:

use college

Create a collection and insert data:

→ db.students.insertOne({name:"Rahul", age:20})

Display the data:

→ db.students.find()

11. Check the PATH

To check MongoDB Server path:

where mongod

Expected:

C:\Program Files\MongoDB\Server\4.4\bin\mongod.exe

To check MongoSH path:

where mongosh

Expected:

D:\Fullstack\mongosh-2.12.0-win32-x64\mongosh-2.12.0-win32-x64\bin\mongosh.exe

12. Stop MongoDB

To stop the MongoDB server, go to the CMD window where **mongod** is running and press:

Ctrl + C

Environment Variable Paths

Add these two paths to the Windows **Path** environment variable:

→ C:\Program Files\MongoDB\Server\4.4\bin

→ D:\Fullstack\mongosh-2.12.0-win32-x64\mongosh-2.12.0-win32-x64\bin

```
C:\Windows\System32>mongod --version
db version v4.4.3
Build Info: {
    "version": "4.4.3",
    "gitVersion": "913d6b62acfbb344dde1b116f4161360acd8fd13",
    "modules": [],
    "allocator": "tcmalloc",
    "environment": {
        "distmod": "windows",
        "distarch": "x86_64",
        "target_arch": "x86_64"
    }
}

C:\Windows\System32>mongosh --version
2.12.0

C:\Windows\System32>net start MongoDB
The requested service has already been started.

More help is available by typing NET HELPMSG 2182.
```

```
C:\Windows\System32>net start MongoDB
The requested service has already been started.

More help is available by typing NET HELPMSG 2182.


C:\Windows\System32>mongosh
Current Mongosh Log ID: 6abdcc75e69ddfbd677d76fc
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.12.0
Using MongoDB:          4.4.3
Using Mongosh:          2.12.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-10-01T07:42:59.284+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
------

test>
```

PRACTICAL NO.7(B)

Aim:- Create a database and collection.

Create/Select a database called collegeDB1:

→ use collegeDB1

Output:-

```
test> use collegeDB1
switched to db collegeDB1
collegeDB1>
```

Create Collection:-

→ db.createCollection("students")

Output:-

```
collegeDB1> db.createCollection("student")
{ ok: 1 }
```

Show all Collections:-

→ show collections

Output:-

```
collegeDB1> show collections
student
students
```

PRACTICAL NO.7(C)

Aim:- Insert records in MongoDB.

Insert One value:-

→db.student.insertOne({

Name:"Rahul",

Age:20,

```
        Course:"BCA",
        City:"Mumbai"
})
```

Output:-

```
collegeDB1> db.student.insertOne({ Name:"Rahul",Age:20,Course:"BCA",City:"Mumbai"})
{
  acknowledged: true,
  insertedId: ObjectId('6abdd464e69ddfbd677d76fd')
}
```

Insert Many values:-

```
→ db.student.insertMany([
        {
                Name:"Priya",
                Age:21,
                Course:"BCA",
                City:"Pune"
        },
        {
                Name:"Amit",
                Age:22,
                Course:"BSc",
                City:"Nashik"
        },
        {
                Name:"Sneha",
                Age:20,
                Course:"BCA",
                City:"Nagpur"
        }
])
```

Output:-

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6abdd684e69ddfbd677d76fe'),
    '1': ObjectId('6abdd684e69ddfbd677d76ff'),
    '2': ObjectId('6abdd684e69ddfbd677d7700')
  }
}
```

PRACTICAL NO.7(D)

Aim:- View records from the collection.

View Record from collection:-

→ db.student.find()

Output:-

```
collegeDB1> db.student.find()
[
  {
    _id: ObjectId('6abdd464e69ddfbd677d76fd'),
    Name: 'Rahul',
    Age: 20,
    Course: 'BCA',
    City: 'Mumbai'
  },
  {
    _id: ObjectId('6abdd684e69ddfbd677d76fe'),
    Name: 'Priya',
    Age: 21,
    Course: 'BCA',
    City: 'Pune'
  },
  {
    _id: ObjectId('6abdd684e69ddfbd677d76ff'),
    Name: 'Amit',
    Age: 22,
    Course: 'BSc',
    City: 'Nashik'
  },
  {
    _id: ObjectId('6abdd684e69ddfbd677d7700'),
    Name: 'Sneha',
    Age: 20,
    Course: 'BCA',
    City: 'Nagpur'
  }
]
```

View a particular record:-

→db.student.find({Name:"Rahul"})

Output:-

```
collegeDB1> db.student.find({Name:"Rahul"})
[
  {
    _id: ObjectId('6abdd464e69ddfbd677d76fd'),
    Name: 'Rahul',
    Age: 20,
    Course: 'BCA',
    City: 'Mumbai'
  }
]
```

View all record in cleaner format:-

→ db.student.find().pretty()

Output:-

```
collegeDB1> db.student.find()
[
  {
    _id: ObjectId('6abdd464e69ddfbd677d76fd'),
    Name: 'Rahul',
    Age: 20,
    Course: 'BCA',
    City: 'Mumbai'
  },
  {
    _id: ObjectId('6abdd684e69ddfbd677d76fe'),
    Name: 'Priya',
    Age: 21,
    Course: 'BCA',
    City: 'Pune'
  },
  {
    _id: ObjectId('6abdd684e69ddfbd677d76ff'),
    Name: 'Amit',
    Age: 22,
    Course: 'BSc',
    City: 'Nashik'
  },
  {
    _id: ObjectId('6abdd684e69ddfbd677d7700'),
    Name: 'Sneha',
    Age: 20,
    Course: 'BCA',
    City: 'Nagpur'
  }
]
```

Update a record:-

→db.student.updateOne({Name:"Rahul"},{$set:{City:"Thane"}})

Output:-

```
collegeDB1> db.student.updateOne({Name:"Rahul"},{$set:{City:"Thane"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Check the updated record:-

→ db.student.find({Name:"Rahul"})

Output:-

```
collegeDB1> db.student.find({Name:"Rahul"})
[
  {
    _id: ObjectId('6abdd464e69ddfbd677d76fd'),
    Name: 'Rahul',
    Age: 20,
    Course: 'BCA',
    City: 'Thane'
  }
]
```

PRACTICAL NO.7(E)

Aim:- Update and delete records.

Delete a specific record:-

→ db.student.deleteOne({Name:"Amit"})

Output:-

```
collegeDB1> db.student.deleteOne({Name:"Amit"})
{ acknowledged: true, deletedCount: 1 }
```

Check if record is deleted:-

→ db.student.find()

Output:-

```
collegeDB1> db.student.find()
[
  {
    _id: ObjectId('6abdd464e69ddfbd677d76fd'),
    Name: 'Rahul',
    Age: 20,
    Course: 'BCA',
    City: 'Thane'
  },
  {
    _id: ObjectId('6abdd684e69ddfbd677d76fe'),
    Name: 'Priya',
    Age: 21,
    Course: 'BCA',
    City: 'Pune'
  },
  {
    _id: ObjectId('6abdd684e69ddfbd677d7700'),
    Name: 'Sneha',
    Age: 20,
    Course: 'BCA',
    City: 'Nagpur'
  }
]
```

PRACTICAL NO.7(F)

Aim:- Connect MongoDB with Node.js.

STEP 1: Open your project folder in VS Code

1. Open Visual Studio Code.
2. Click:

   File → Open Folder
3. Select this folder:

   C:\Users\user\mongodb-node-practical13
4. Click "Select Folder".

5. You should see these files/folders in the VS Code Explorer:
   - node_modules
   - package-lock.json
   - package.json

STEP 2: Create server.js

1. In VS Code, look at the Explorer panel on the left.

2. Right-click the project folder:
   Mongodb-node-practical13

3. **Select:**
   New File

4. **Enter:**
   server.js

5. Press Enter.

6. Open server.js.

STEP 3: Check your project structure

Your project should now look like:

```
mongodb-node-practical13
│
├── node_modules
│
├── package-lock.json
│
├── package.json
│
└── server.js
```

STEP 4: Add MongoDB connection code

Copy the following code into server.js:

```
const { MongoClient } = require("mongodb");

const uri = "mongodb://127.0.0.1:27017";

const client = new MongoClient(uri);

async function connectDB() {
   try {
      await client.connect();

      console.log("MongoDB connected successfully!");

      const db = client.db("practical13");

      console.log("Database selected:", db.databaseName);
   } catch (error) {
      console.error("MongoDB connection failed:", error);
   }
}

connectDB();
```

STEP 5: Make sure MongoDB Server is running

MongoDB must be running before you execute server.js.

If MongoDB was installed as a Windows service, open Command Prompt and check:

net start MongoDB

If it says that the MongoDB service is already started, that is fine.

If MongoDB is not installed/running as a Windows service, start MongoDB using the MongoDB installation method you used.

STEP 6: Open the VS Code terminal

In VS Code:
Terminal → New Terminal
The terminal should open at your project folder.

It should look similar to:
PS C:\Users\user\mongodb-node-practical13>

STEP 7: Run server.js

In the VS Code terminal, type: **node server.js**

Press Enter.

STEP 8: Check the output

If everything is working correctly, you should see:

MongoDB connected successfully!
Database selected: practical13

This means Node.js successfully connected to your local MongoDB server.